Find the Promos — solution doc

Code: github.com/karangupta23/promo-finder · **Demo:** promofinder-web.vercel.app · **Raw outputs:** linked in the submission form · **Show:** *I Married a Broken Soldier. He was a Duke* (50 episodes, 103 min, 3,727 screenplay blocks)

1. Problem and why

I picked Problem 1 because it has real input, an output that can be checked exactly (episode + timestamps), and a clear human baseline: today an editor does it by hand. Every new show needs a stream of 20–60 s paid-social promos, and today an editor watches every episode to find them. The tool reads a show's screenplays and returns a ranked shortlist of **continuous clips** — episode, exact start/end, the full blocks in the clip, the hook (first 3 s), the cliffhanger and the question it leaves open, why it works for a first-time viewer, scores, risks — plus a separate posting order. It is a reusable Python CLI (`python run.py all --input <folder|zip>`); the website only displays frozen results.

2. Assumptions

- Single continuous clips of 20–60 s; no stitching (the brief allows it, I scoped it out).
- The screenplay stands in for the video: "first 3 s" means what the script times there.
- A promo must work for a stranger, so the judges see **only the clip** — no loglines, continuity notes or later scenes.
- Original speaker labels are kept (the pack calls the same woman `JULIANA` and `LADY JULIANA`).
- The 3 s hook / 5 s cliffhanger windows are practical rules for this assignment, not universal truths.

3. How it works

`parse → candidates → Stage A → Stage B → rank → payoff-leakage check → posting order`

1. **Parse.** Every block gets a stable id (`E03_B067`), original timestamps, type and speaker. Problems are flagged, never fixed: 50/50 episodes, 0 invalid timestamps, 10 overlapping lines, 1 out-of-scene block.
2. **Candidates — the decision that matters most:** a clip never generated can't be picked. Two independent families: *shot-aligned* (pick the strongest endings, then the best camera-cut opening 20–60 s earlier, plus one keyword-free window per episode) and *event-anchored* (open on a sharp line or a slap/entrance, even mid-shot). Never cut inside a line of dialogue. **333 candidates** (4–7 per episode, mean 33 s); settings frozen as `cand-v1`.
3. **Stage A** (Gemini 3.5 Flash-Lite, every candidate). PASS / UNCERTAIN / FAIL plus 0–10 scores with evidence block ids for hook, cliffhanger, standalone comprehension, emotion, dead time, risks. **Code checks the model's claims:** cited ids must be in the clip, the hook evidence in the first 3 s, the cliffhanger evidence near the end, otherwise the verdict is capped at UNCERTAIN (3 of 333 clips tripped a check).
4. **Stage B** (Gemini 3.6 Flash, 18 clips: 12 top, 4 uncertain, 2 the keyword baseline liked but Stage A failed). A skeptical editorial pass: central character, what they want, what changes, the open question, missing context, "reveals too much?", "payoff too early?", a tighter re-cut.
5. **Rank.** Hard gates (hook, cliffhanger and standalone must all PASS; any FAIL rejects; the rest go to human review) → weighted score (hook 25, cliffhanger 25, standalone 20, emotion 15, pacing 10, clean cut 5) → drop clips sharing > 50 % with a better clip from the same episode. The **posting order** is separate: ≤ 2 per episode in the first 10, no back-to-back repeats, a bonus for episodes 1–10 so a viewer who clicks starts near the beginning.
6. **Payoff-leakage check** (Flash-Lite, top 15). The only step allowed to see the ~20 s *after* the clip, to catch clips that answer their own question. Kept separate so judges never use future information.

Human in the loop: clips with any UNCERTAIN gate go to an editor's review queue instead of the shortlist, and an editor still watches every shortlisted cut before posting; the tool replaces the scrubbing through 100 minutes of footage, not the final call. Model-agnostic by design: Gemini or any OpenAI-compatible endpoint via one flag, prompts are versioned files, every call is cached by provider + model + prompt version + prompt.

4. Scale, calls, runtime, cost

Stage	Model	API calls	Input / output tokens	Model time
Parse + candidates	—	0	—	~3 s
Stage A	gemini-3.5-flash-lite	340 (333 + 7 retries)	549k / 222k	~38 min
Stage B	gemini-3.6-flash	18	32k / 21k (+24k thinking)	~5 min
Leakage	gemini-3.5-flash-lite	15	30k / 4k	~2 min

0 failed calls, 7 rate-limit retries (backoff honouring the server's retry hint). Cost: **$0** on the Gemini free tier; under US$1 at paid rates. The free tier's binding limit is Flash at ~20 requests/day, which is why Stage B is budgeted (18 by default, `--budget 40+` on a paid key). Per new show: ~7 Flash-Lite calls per episode, ~45 min unattended, fully resumable from cache.

Funnel: 333 candidates → 153 after overlap removal → **48 shortlisted** (all gates PASS), 63 for human review, 42 rejected; Stage A on its own passed all three gates for 61. Stage A's main filters were the cliffhanger (114 FAIL) and standalone comprehension (219 UNCERTAIN); its hook verdict was lenient (252 PASS).

5. Does it work?

All three methods rank the **same frozen candidate pool**; prompts were fixed before the evaluation run; episodes 1–5 were used for development and 6–50 for evaluation. Each method's top 10 on episodes 6–50 plus 4 lower-ranked clips per method (30 clips) were pooled, shuffled and labelled Pass / Uncertain / Fail on hook, cliffhanger, comprehension and editing, without method labels.

Method	Pass	Uncertain	Fail	precision@10	Pass + Uncertain	Comprehension passes
Keyword baseline	7	2	1	**0.70**	0.90	10 / 10
Stage A (Flash-Lite)	7	3	0	**0.70**	1.00	7 / 10
Stage A + B (Flash)	5	5	0	**0.50**	1.00	6 / 10

What this says. The LLM judges never picked an unusable clip (0 Fail; at worst a pick needed a re-cut) and their cliffhangers were the strongest (10/10 passed), but they did **not** beat the keyword baseline on postable-as-is. Their picks more often needed backstory: the judges' standalone verdicts were too generous for mid-season clips, and Stage B — which passed 17 of 18 clips it saw — re-ordered toward exactly those. The baseline did well partly because the scripts label their own cliffhangers, which its keyword list rewards (see failure 2). Lower-ranked sample (ranks 11–40): 1–2 of 4 postable per method and none failed, so the pool holds more usable promos than the top 10 — ranking, not candidate generation, is the bottleneck. With 10 clips per method a 2-clip gap is not conclusive. Next step: make comprehension a stricter gate (judge must name the conflict from inside the clip) and make Stage B able to fail clips.

6. Where it fails (real cases from this run)

1. **The cliffhanger is the payoff — Ep 3, 00:58.30–01:25.50.** Stage A's #1 clip (91.8), passed by Stage B: a stranger stops guards dragging Juliana out, and the clip ends on his knockout punch. The question "can he stop them?" is answered on screen. Only the leakage check caught it. *Better cut:* end at 01:23.90 on "…let her go." (25.6 s). *Fix:* feed leakage TRIMs back in as new candidates; tell judges a confrontation resolved inside the clip is a payoff.
2. **Stage B believed the script's own word "cliffhanger" — Ep 16, 00:48.80–01:23.19.** Stage A correctly failed a clip ending on a tender, resolved moment; Stage B overruled it, citing a sound cue "swelling to a dramatic cliffhanger hold". The better cut of the same scene (00:25.80–00:54.40, ending on the betrayal reveal "Gerald…") was in the pool but ranked #25. *Fix:* strip the screenplay's self-descriptions before judging; require cliffhanger evidence from the last line or action, not music.
3. **Posting order: the early-episode bonus outweighed quality — Ep 1.** Five of ten posting slots went to clips from episodes 1–5 at quality ranks 17–26 that Stage B never reviewed; two episode 1 clips share 11 s of footage. *Fix:* make the bonus a ≤ 1-point tie-breaker among Stage B-confirmed clips; treat any shared footage as a conflict.

Limits: judges read the script, not the edit (no faces, performances or real pacing); keyword lists that seed candidates were written for this show; Stage B covers a small, budgeted subset on the free tier; temperature 0 reduces but doesn't remove run-to-run variation.

7. AI tools used, and one place they got it wrong

I used Claude to design and write the pipeline and tests, and GitHub Copilot in VS Code to set up the local environment and run it. **A real miss:** Copilot reported the full run as "currently running" when it had actually stopped at once — it was launched without `--input` while the scripts sat in a folder the tool doesn't look in by default. Caught by checking for the run's output files rather than trusting the message; fixed by renaming the folder to `pack/`. Lesson: verify progress from artefacts (files, logs), not from an assistant's summary.